

Sistemas Operativos

Memoria Virtual y Paginación

Apunte de clase — Ariel Godio

Este documento es una versión narrativa completa de la clase teórica sobre memoria virtual y paginación. Incluye todos los conceptos, problemas, analogías, mecanismos y discusiones presentados durante la clase.

Contenidos

1. Reflexión inicial: lo que damos por sentado
2. Responsabilidades del sistema operativo
 - 2.1. Abstracción
 - 2.2. Interposición
3. El mundo sin abstracción ni interposición — problemas
 - 3.1. Colisión entre procesos
 - 3.2. Protección del sistema operativo
 - 3.3. Limitación de memoria física (swap)
 - 3.4. Dificultad para ejecutar programas parcialmente cargados
 - 3.5. Redundancia de librerías compartidas
 - 3.6. Dependencia de direcciones absolutas
 - 3.7. Fragmentación de memoria
 - 3.8. Imposibilidad de establecer permisos por región
4. La solución: Memoria Virtual
 - 4.1. Concepto central
 - 4.2. Analogía con el sistema telefónico
 - 4.3. Espacio de direcciones virtuales vs. físicas
5. Implementación de la memoria virtual
 - 5.1. La MMU (Memory Management Unit)
 - 5.2. El kernel mapeado en todos los procesos
 - 5.3. El CPL (Current Privilege Level) como guardián
 - 5.4. Librerías compartidas: permisos virtuales independientes
 - 5.5. El TLB (Translation Lookaside Buffer)
 - 5.6. Linker estático vs. dinámico
6. Paginación
 - 6.1. Concepto de página y marco de página
 - 6.2. Tabla de páginas
 - 6.3. Traducción de direcciones virtuales a físicas
 - 6.4. Page Fault (fallo de página)
7. Beneficios que resuelven los problemas originales
8. Temas pendientes para la próxima clase

1. Reflexión inicial: lo que damos por sentado

La clase comienza con una reflexión fundamental: desde el momento en que escribimos nuestro primer programa **Hello World**, jamás nos preocupamos por resolver el problema de en qué parte de la memoria física había que ubicar cada segmento del programa.

Sin embargo, un programa en C tiene muchas partes que necesitan residir en memoria:

- **Segmento de código:** las instrucciones del programa compiladas.
- **Stack:** necesario para las llamadas a funciones (cuando llamamos a printf o a main).
- **Heap:** memoria dinámica asignada con malloc.
- **Read-only data (rodata):** datos de solo lectura, como los strings literales.
- **Librerías:** el código de printf vive en libc, que también ocupa memoria.

Nunca pensamos en nada de esto. Lo dimos por hecho. El objetivo de esta clase es, en palabras del profesor: **"perderlo por un rato para entender lo que tenemos"**. Es el clásico: uno no sabe lo que tiene hasta que lo pierde.

2. Responsabilidades del sistema operativo

Un sistema operativo tiene dos responsabilidades claramente diferenciadas, y las cumple mediante dos mecanismos distintos:

2.1. Abstracción

Una **abstracción** es una simplificación de la realidad. Oculta detalles complejos y nos presenta una interfaz simple para interactuar con ellos. Cuando escribimos en disco usando un file descriptor, no necesitamos conocer las 400 páginas del manual del controlador SATA. El sistema operativo nos abstrae de esa complejidad y nos da una interfaz limpia: open, write, close.

Analogía: es como la creación de la vida. Desde afuera parece simple: se unen 2 células y aparece un ser humano. Pero la cantidad de mecanismos moleculares y biológicos que ocurren en el proceso es extraordinariamente compleja. La abstracción nos permite no preocuparnos por esos detalles.

2.2. Interposición

La **interposición** es el mecanismo mediante el cual el sistema operativo se interpone entre los procesos y los recursos físicos, tomando el control de las decisiones de administración. Un proceso no puede apropiarse del CPU, del disco ni de la memoria por su cuenta: debe pasar por el sistema operativo.

Analogía 1 — los padres y el dinero: si los padres son los que administran el presupuesto familiar y los hijos son los que gastan, los padres se interponen entre los gastos de los hijos y el dinero disponible, tomando las decisiones de asignación.

Analogía 2 — las aulas del ITBA: si cada materia pudiera tomar unilateralmente el aula que quisiera, habría colisiones permanentes. La institución (el sistema operativo) se interpone: uno debe solicitar aula a través de un canal oficial (una syscall), con permisos adecuados. La institución también abstrae los detalles físicos de las aulas: el profesor no necesita conocer todas las sedes ni qué paredes son móviles.

El TP2 de la materia es un ejemplo de **sin interposición**: el kernel provee algunas abstracciones (procesos, semáforos) pero no se interpone en el acceso a memoria. Cualquier proceso puede acceder a cualquier dirección física. Esto sirve pedagógicamente para entender los problemas que genera la ausencia de interposición.

3. El mundo sin abstracción ni interposición — problemas

En este escenario, cada proceso accede **directamente** a la memoria física. Veamos todos los problemas que esto genera:

3.1. Colisión entre procesos

Cada proceso decide unilateralmente qué región de memoria utilizar. Si ls quiere usar cierta región para su heap y bash quiere usar la misma región para su stack, habrá una colisión: uno terminará pisando la memoria del otro y ambos fallarán. No hay manera de coordinarlo sin una autoridad central.

3.2. Protección del sistema operativo

El propio sistema operativo es código en memoria. ¿Quién lo protege? Un stack overflow accidental de cualquier proceso podría pisar el código del kernel, corrompiendo todo el sistema. Ni hablar de ataques maliciosos intencionales.

3.3. Limitación de memoria física (swap)

Sin un mecanismo de gestión, es muy difícil ejecutar programas que necesiten más memoria que la que tenemos físicamente instalada. Si tengo 4 GB de RAM y mi proceso necesita 8 GB, sin interposición no hay forma de resolver esto.

La solución clásica es el **swap**: bajar páginas de memoria al disco para hacer lugar. ¿Por qué funciona? Porque en un instante dado, un proceso solo necesita unos pocos bytes (los de la instrucción en ejecución, sus operandos y un poco más). No existe ninguna instrucción que requiera tener 17 GB de RAM disponibles simultáneamente. Entonces podemos mantener en RAM solo lo que se está usando ahora, y mover el resto al disco.

*Historia: este problema es antiquísimo. En los años 60 existían los **overlays**: programas divididos en secciones que el propio programador cargaba manualmente. En los 90, los juegos venían en múltiples disquetes, y el usuario debía insertar el disco correcto en cada momento del juego (Monkey Island, por ejemplo). Hoy en día existen los DLC (contenido descargable). La diferencia con la memoria virtual moderna: antes era el **programador** quien gestionaba esto manualmente; hoy lo hace el kernel **automáticamente y de forma transparente**.*

3.4. Dificultad para ejecutar programas parcialmente cargados

Sin memoria virtual, un programa tiene que estar completamente cargado en memoria para poder ejecutarse. El motivo: si el código contiene una instrucción **jmp 0x100** y esa dirección no está en memoria (está en disco), el procesador saltará a una región de memoria que contiene basura, con resultados catastróficos.

Esto es molesto además porque fuerza a cargar todo el código del programa aunque solo se vaya a usar una pequeña fracción. Por ejemplo, Chomchamps detecta si no le pasaron el parámetro -p y hace exit inmediatamente: ¿tiene sentido cargar todo el código del juego para eso?

3.5. Redundancia de librerías compartidas

Si Bach y Ls necesitan libc, sin gestión centralizada cada uno cargará su propia copia en memoria. Con 500 procesos corriendo simultáneamente (algo perfectamente normal en una computadora moderna), eso significa 500 copias de libc en memoria, aunque libc ocupa apenas 1-2 MB, son casi 1 GB desperdiciados solo en una librería.

***Analogía:** sería como si el profesor tuviera que mandarle un email individual a cada alumno con el anuncio, en lugar de publicar un único anuncio en el campus que todos puedan ver simultáneamente.*

3.6. Dependencia de direcciones absolutas

Cuando compilamos un programa, el compilador embebe direcciones físicas concretas en el código. Por ejemplo, en el ensamblador de un Hello World, la instrucción que carga el parámetro de printf puede ser: **mov rdi, 0x402000**, donde 0x402000 es la dirección física donde debe estar el string "Hello World".

Si el programa no se carga exactamente en el lugar para el que fue compilado, esa dirección 0x402000 apuntará a basura y el programa fallará. Esto obliga a cargar cada programa siempre en la misma posición de memoria física, lo cual hace casi imposible ejecutar múltiples programas simultáneamente.

***Analogía del empleado del supermercado:** una remera que dice "Hola, estoy para ayudarte" solo funciona cuando el empleado está dentro del supermercado. Si lo ves caminando por la vereda con esa remera, el contexto está equivocado y no funciona. El programa funciona solo si está cargado exactamente donde fue diseñado.*

3.7. Fragmentación de memoria

Con múltiples procesos administrando memoria por su cuenta, la memoria tiende a fragmentarse: hay memoria libre total suficiente para una tarea, pero distribuida en pequeños bloques no contiguos que no sirven para alojar un bloque grande.

***Analogía del bar/restaurant:** tengo 5 mesas de 2 sillas, hay 6 personas sentadas (de a 1 por mesa). Llegan 2 parejas. Hay 4 sillas libres en total, pero distribuidas de a 1: ninguna pareja puede sentarse junta. El mozo de Güerrin que le dice a alguien "esa mesa no" cuando viene solo a una mesa para 4 entiende perfectamente el concepto de fragmentación: está preservando espacio contiguo.*

3.8. Imposibilidad de establecer permisos por región

Sería deseable poder definir permisos por tipo de memoria:

- **Código:** solo lectura y ejecución (no escritura).
- **Rodata:** solo lectura.
- **Stack:** lectura y escritura (no ejecución).
- **Heap:** lectura y escritura.

Sin interposición, cualquier proceso puede leer, escribir o ejecutar cualquier dirección de memoria sin restricción.

Analogía del museo: sería ideal que las piezas en exposición sean read-only (nadie puede tocarlas ni modificarlas), el libro de quejas sea read-write, y el depósito privado del museo sea completamente inaccesible para los visitantes.

4. La solución: Memoria Virtual

4.1. Concepto central

La memoria virtual resuelve todos los problemas anteriores mediante una idea elegante: **separar el espacio de direcciones virtuales del espacio físico.**

Cada proceso tiene su **propio espacio de direcciones virtuales**, completamente privado. Cuando un proceso accede a la dirección 0x1000, no está accediendo directamente a la posición física 0x1000 de la RAM. En cambio, existe una capa de traducción que mapea esa dirección virtual a alguna dirección física.

El proceso ejecuta creyendo que es el dueño exclusivo de toda la memoria. Esta es la misma ilusión que el scheduler crea para el CPU: cada proceso cree que es el único en ejecución y que tiene el CPU para él solo. Aquí, cada proceso cree que tiene toda la memoria para él solo.

Dos procesos pueden usar la misma dirección virtual (por ejemplo, 0x1000) sin conflicto, porque cada una mapea a una dirección física diferente. Bash en 0x1000 puede ser una variable A, y Ls en 0x1000 puede ser una variable B completamente distinta.

4.2. Analogía con el sistema telefónico

Para entender los espacios de direcciones, el profesor hace una analogía muy completa con el sistema de telefonía:

Cuando marcamos un número de teléfono, no nos conectamos directamente a ningún cable físico. Una **central telefónica** (el sistema operativo) se interpone y toma decisiones. El conjunto de todos los números que podemos marcar es nuestro **espacio de direcciones**.

Analogía telefónica	Equivalente en memoria
Número de teléfono	Dirección virtual

Central telefónica	Sistema operativo / MMU
Abonado (persona real)	Dirección física de RAM
Diferentes números → mismo teléfono	Diferentes virtuales → misma física (aliasing)
911 → distintos servicios según ubicación	Mismo virtual → distintas físicas por proceso
Abortar llamada: sin saldo / restricción judicial	Violación de permisos / CPL insuficiente
Número no existe (no hay abonado)	Dirección virtual sin mapeo → Segfault
Línea ocupada → esperar	Página no presente → Page Fault (esperar a disco)
Pizzería se mudó, mismo número funciona	Página movida en física, virtual sigue igual

4.3. Espacio de direcciones: características clave

Único por proceso: el espacio de direcciones es privado para cada proceso. La dirección 0x1000 en bash y la dirección 0x1000 en ls son mundos completamente independientes. Son como marcar el mismo número desde distintos teléfonos: el resultado puede ser diferente.

Virtual ≠ Físico: cualquier puntero que escribamos en un programa C es una dirección virtual. Si hacemos `printf("%p", ptr)`, lo que se imprime es la dirección virtual, nunca la física.

No almacenar punteros virtuales en memoria compartida: como los espacios de direcciones son privados, una dirección virtual 510 en Chomchamps no corresponde a la misma posición que 510 en la Vista. Por eso, dentro de la memoria compartida se deben usar **offsets** (desplazamientos relativos al inicio del bloque compartido), no punteros absolutos.

5. Implementación de la memoria virtual

5.1. La MMU (Memory Management Unit)

La interposición que provee la memoria virtual es **completa**: cada acceso a memoria, sin excepción, es supervisado. Esto solo es posible con ayuda del hardware, porque el kernel es software: mientras un proceso está ejecutando, el kernel **no está ejecutando** (hay un solo CPU, o un solo núcleo). El kernel no puede vigilar activamente cada instrucción.

La solución: el kernel **configura el hardware** una vez durante el arranque, diciéndole: "a partir de ahora, cada acceso a memoria debe ser verificado". Luego, el hardware (la **MMU — Memory Management Unit**) se encarga autónomamente de verificar cada acceso. Si algo sale mal, la MMU genera una excepción que devuelve el control al kernel.

La MMU verifica en cada acceso: si la dirección virtual tiene un mapeo a memoria física, si el proceso tiene permisos para hacer la operación solicitada (leer/escribir/ejecutar), y si el nivel de privilegio (CPL) es suficiente. El kernel configura todo esto mediante la **IDT (Interrupt Descriptor Table)**: una tabla que le dice al hardware qué función ejecutar ante cada tipo de excepción.

5.2. El kernel mapeado en todos los procesos

Una pregunta natural: si el kernel es software y debe ser llamado cuando un proceso hace una syscall, ¿cómo puede el proceso llamarlo si el código del kernel no está en su espacio de memoria?

La respuesta: el **código del kernel está mapeado en el espacio de direcciones de todos los procesos**, siempre en la misma dirección virtual (generalmente las direcciones más altas). Una syscall no es más que una **llamada a función con pasos extra**: el proceso voluntariamente salta al código del kernel mapeado en su espacio virtual y lo ejecuta.

Es el proceso quien ejecuta la syscall, no el kernel. El kernel solo provee el código. Esto es completamente transparente para el programador: llamar a `write()` se parece exactamente a llamar a `strlen()`; la diferencia es interna.

No solo el código del kernel está mapeado, sino también sus **estructuras de datos**. Por ejemplo, para ejecutar `getpid()` (devolver el PID del proceso actual), la syscall necesita leer el PCB (Process Control Block) del proceso, que es una estructura del kernel.

5.3. El CPL (Current Privilege Level) como guardián

Si el código y las estructuras del kernel están mapeados en el espacio de cada proceso, ¿qué impide que un proceso malicioso acceda directamente a esas estructuras y las modifique?

La respuesta es el **Current Privilege Level (CPL)**. Cada página de memoria tiene, además de los permisos de lectura/escritura/ejecución, un **nivel de privilegio mínimo requerido** para acceder a ella.

La memoria del kernel (código y estructuras) requiere CPL = 0 (modo kernel). Un proceso de usuario corre con CPL = 3. Si intenta acceder a memoria protegida con CPL = 0, la MMU genera inmediatamente una excepción.

El único mecanismo que eleva el CPL de 3 a 0 es una **syscall**. Pero las syscalls solo ejecutan código escrito por el kernel, no código arbitrario del proceso. Es un mecanismo controlado: el proceso puede pedirle al kernel que haga cosas en su nombre, pero no puede directamente manipular las estructuras internas.

Mientras se ejecuta una syscall (CPL = 0), el kernel sí puede acceder a la memoria del proceso (stack, heap, etc.) porque esa memoria tiene CPL requerido = 3 y el kernel tiene CPL = 0, que es más privilegiado (menor número = más privilegio).

Analogía: conocés el número de teléfono de tu ex. Pero hay una restricción judicial: CPL no suficiente. Conocer la dirección virtual no sirve de nada si no tenés el privilegio necesario.

5.4. Librerías compartidas: permisos virtuales independientes

La memoria virtual permite que dos procesos **compartan un único bloque de memoria física** (como `libc`) pero con **permisos definidos independientemente** en el espacio virtual de cada uno. Los permisos se definen a nivel de mapeo virtual, no de la página física.

Esto es exactamente lo que ocurre en Chomchamps: el estado del juego es una única memoria compartida. El master la mapea con permisos de lectura **y escritura**. Los players

y la vista la mapean con permisos de **solo lectura**. El bloque físico es el mismo, pero cada proceso tiene sus propios permisos en su espacio virtual.

5.5. El TLB (Translation Lookaside Buffer)

Un problema de eficiencia: si la tabla de páginas está almacenada en la RAM, traducir cada dirección virtual requeriría un acceso a RAM (para leer la tabla) y luego otro acceso a RAM (para acceder al dato). Esto reduciría la eficiencia a la mitad.

La solución es el **TLB (Translation Lookaside Buffer)**: una caché de traducciones recientes almacenada directamente dentro de la MMU (hardware). Si la traducción virtual→física está en el TLB, no hace falta ir a RAM para buscarla.

El hecho de que el kernel esté mapeado en la **misma dirección virtual en todos los procesos** tiene una razón de eficiencia directamente relacionada con el TLB: cuando se hace un context switch, las entradas del TLB correspondientes al kernel no necesitan invalidarse (siguen siendo válidas para el nuevo proceso). Esto evita un costo importante de recargar el TLB en cada cambio de proceso.

El registro **CR3** apunta a la tabla de páginas del proceso actual. Cuando se realiza un context switch, se actualiza CR3 para apuntar a la tabla del nuevo proceso. Pero las entradas del TLB correspondientes al kernel no se invalidan.

5.6. Linker estático vs. dinámico

Al final de la clase el profesor menciona dos formas de incluir librerías en un programa:

Linker estático: el código de la librería se incorpora directamente dentro del binario del programa. Cada proceso tiene su propia copia del código de la librería. No hay compartición posible.

Linker dinámico: el binario solo contiene referencias a las librerías. En tiempo de ejecución, el **loader** (que se ejecuta antes del main del programa) resuelve esas referencias cargando las librerías compartidas en memoria y mapeándolas en el espacio virtual del proceso. Múltiples procesos pueden compartir la misma copia física de la librería. Si usamos strace con ls, podemos ver todas estas llamadas a mmap, open, etc. que ocurren antes de que el programa empiece a listar archivos.

6. Paginación

Hasta ahora hablamos de memoria virtual en términos abstractos: "páginas de memoria", "cachos", "áreas". La paginación le da una **granularidad concreta y uniforme** a todo esto.

6.1. Concepto de página y marco de página

Una **página** es un bloque de bytes contiguos de tamaño fijo. El tamaño clásico es **4 KB** (4096 bytes). Todo el espacio de direcciones (tanto virtual como físico) se divide en bloques del mismo tamaño.

La terminología estándar es: en el espacio virtual se habla de **páginas virtuales**, y en el espacio físico se habla de **marcos de página** (page frames). Generalmente tienen el mismo tamaño por conveniencia, aunque no es estrictamente necesario.

Definir permisos byte a byte sería innecesario y extremadamente costoso en términos de espacio de almacenamiento de metadatos. La paginación permite definir permisos a nivel de página: toda la página tiene los mismos permisos.

6.2. Tabla de páginas

La **tabla de páginas** es la estructura de datos que implementa la función de mapeo de virtual a físico. Es esencialmente un arreglo: dado el número de página virtual como índice, devuelve el número de marco de página físico correspondiente.

Cada proceso tiene su propia tabla de páginas (porque cada proceso tiene su propio espacio de direcciones virtuales). El registro **CR3** del procesador contiene la dirección de la tabla de páginas del proceso actualmente en ejecución.

Cada entrada de la tabla contiene, al menos: el número de marco de página físico y un **bit de presencia** que indica si la página está actualmente en RAM o en disco.

6.3. Traducción de direcciones virtuales a físicas

Con páginas de 4 KB, se necesitan exactamente **12 bits** para identificar cualquier byte dentro de una página ($2^{12} = 4096 = 4 \text{ KB}$). En una arquitectura de 64 bits, los bits de una dirección se dividen:

Bits [11:0] → Offset dentro de la página. Estos 12 bits pasan directamente a la dirección física SIN traducción. El byte número 47 dentro de una página virtual es el byte número 47 dentro del marco de página físico correspondiente.

Bits superiores [63:12] → Número de página virtual. Estos bits se usan como índice en la tabla de páginas para obtener el número de marco físico.

La dirección física final se construye concatenando: el número de marco de página obtenido de la tabla + los 12 bits de offset originales.

Regla mnemotécnica para los bits: $2^{10} = 1K$, $2^{20} = 1M$, $2^{30} = 1G$, $2^{40} = 1T$. Por tanto: $4K = 2^{12} \rightarrow 12 \text{ bits de offset}$.

6.4. Page Fault (fallo de página)

Un **page fault** ocurre cuando se intenta acceder a una dirección virtual cuya página no está presente en memoria física (el bit de presencia en la tabla de páginas está a 0).

La secuencia de eventos cuando ocurre un page fault:

1. El proceso intenta acceder a una dirección virtual cuya página no está en RAM.
2. La MMU detecta que el bit de presencia es 0 y genera una **excepción** (page fault).
3. El hardware transfiere el control al kernel mediante la IDT.

4. El kernel **bloquea el proceso** (esperar en disco toma microsegundos o más).
5. El kernel encuentra un marco de página libre (o libera uno bajando una página a disco).
6. El kernel lee la página desde disco a ese marco.
7. El kernel actualiza la tabla de páginas: el bit de presencia pasa a 1, y se registra el número de marco donde está ahora la página.
8. El kernel **despierta el proceso** (lo vuelve a poner en estado READY).
9. El proceso **reintenta la misma instrucción**. Las excepciones no avanzan el instruction pointer (a diferencia de las interrupciones), por lo que la instrucción se reintenta automáticamente. Esta vez, la página está presente y la instrucción ejecuta normalmente.

Esta transparencia es el corazón de la memoria virtual: el proceso **no sabe que hubo un page fault**. Sencillamente la instrucción tardó un poco más. El programador que escribió Bach no tuvo que implementar nada de esto: lo obtiene gratis del sistema operativo.

Importante: un page fault no siempre indica un error. Puede ocurrir por múltiples razones: la página fue movida a disco (swap), la página nunca fue asignada en RAM (lazy allocation), o implementación de copy-on-write. Un page fault que sí indica error termina en un segmentation fault y el proceso es matado.

7. Beneficios que resuelven los problemas originales

Problema	Solución con memoria virtual
Colisión entre procesos	Cada proceso tiene su espacio virtual privado
Protección del kernel	CPL + permisos de página protegen la memoria del kernel
Limitación de RAM	Swap: páginas no usadas se mueven a disco transparentemente
Programa cargado parcialmente	Page fault: el kernel carga bajo demanda, transparente
Redundancia de librerías	Librerías compartidas: una copia física, múltiples virtuales
Dependencia de direcciones absolutas	Cada proceso tiene su propia tabla: la virtual siempre es la misma
Fragmentación	Fragmentos físicos no contiguos se mapean a virtual contiguo
Imposibilidad de permisos	Permisos por página + CPL en cada acceso

8. Temas pendientes para la próxima clase

La clase termina con el inicio del tema de tabla de páginas. Para la próxima clase quedan:

- Más detalle sobre page faults y todos los escenarios posibles.
- TLB en profundidad (Translation Lookaside Buffer).
- Tablas de páginas multinivel.
- Tablas de páginas invertidas.

- Algoritmos de reemplazo de páginas.
- Administración de memoria libre.

"Uno no sabe lo que tiene hasta que lo pierde. Por eso lo perdimos por un rato." — Ariel Godio